Agent Orchestrator: final engineering summary

Describes version 0.8.1. The design is in architecture.md; every requirement of the brief is mapped to code and evidence in requirements-traceability.md. This document does not repeat them: it says what was done and why, what supports the claims, and where the limits are.

1. At a glance

What was built. An agentic orchestration layer that takes a requirement through plan, design, code, tests, documentation, UI, review and release readiness, under explicit controls: a dependency graph with entry and exit gates, parallel stages with a join, human approvals, bounded retries, fallback, rollback and safe-stop, named security, compliance and change-control policies, a hash-chained audit log, reliability metrics, and automatic re-planning when an upstream output changes. A URL shortener is the system it builds (greenfield), fixes (brownfield, BUG-142) and changes from a vague request (ambiguous, "make links expire sensibly"). A local admin page runs everything, including the human decisions.

Headline results.

- 39 named controls and 24 audit event types, all generated into the reference documentation from the code.
- 24 scripted faults, each stopped by a named control: 22 recover after one retry, 2 repeat and halt safely.
- 363 test functions (413 cases, because some are parameterised and the generated service's tests run individually); the key guards were also deliberately broken to confirm the tests notice.
- Three real bugs found and fixed only by running on Windows.

Honest status. Agents are mocks (stored, reviewed artifacts) unless a real model is chosen. That proves the *governance*, not the quality of a model's generation. **The author has run the real-model path live (Sonnet): the first runs exposed four gaps the mock could not (section 4), all fixed, and a full run of the greenfield shortener has since completed. One run is an existence proof, not a measure of reliability.**

2. Plan and rationale

The brief names workflow orchestration the critical differentiator, so the plan put the orchestration engine first, then scenarios of rising difficulty, and only then the conveniences.

Version	What it added	Why in this order
0.1.0	Engine, retries, fallback, rollback, safe-stop, approvals, audit chain, metrics, policies, the greenfield shortener	The differentiator first; a vertical slice that already runs end to end
0.2.0	Web UI stage; the brownfield scenario with code-impact analysis	Existing code is harder than new code, and the UI exercises a second kind of security policy
0.3.0	Automatic re-planning, a planner stage, the ambiguous scenario with human decisions	The hardest requirements (non-linear, dynamic, human-in-the-loop) once the base was trusted
0.4.0	A real-model client behind the same interface	Proves the governance is not tied to the mock
0.5.x	Local admin page, one-click service launch, Windows fixes	One window for oversight; real use exposed real bugs
0.7.x to 0.8.0	Size limits, a grounded reviewer, review-driven rework	The author's first live runs showed what mock agents could not
later	Architecture overview, traceability, this summary	Describe the system once it stopped changing

Key choices (reasons and rejected alternatives: architecture.md section 11): a custom engine, policy as code, the standard library only, and mock agents by default (offline, deterministic, free).

3. Artifacts

Artifact	Where
Orchestration engine	`orchestrator/core/`
Governance: policies, gates, sandbox, code analysis	`orchestrator/governance/`
Agents and the LLM layer (mock, real, hybrid)	`orchestrator/agents/`, `orchestrator/llm/`
Audit log and metrics	`orchestrator/observability/`
The workflows and the mock agents' reference artifacts	`orchestrator/workflows/`
The generated shortener: service, tests, documentation, UI	`orchestrator/workflows/reference/shortener/`
Command line and runner	`orchestrator/cli.py`, `orchestrator/runner.py`
Admin page and service launcher	`orchestrator/admin/`
Tests	`tests/`
Documentation, decisions, scenario write-ups	`docs/` (start with architecture.md)
Output of every run (audit log, metrics, report, artifacts)	`runs/<id>/`, created when something runs

4. Validation evidence

Tests by area.

Area	What it checks	Test functions
Engine, resilience and re-planning	graph execution, retries, fallback, rollback, safe-stop, fingerprints, revisions	77
Policies and governance	security, compliance, change-control, planning and clarification policies; code analysis	92
The three scenarios	the greenfield, brownfield and ambiguous workflows end to end, including injected faults	65
The generated service and its UI	the shortener over real HTTP, its web UI (script harness), the live smoke test	19
Real-model integration	the Anthropic client against a fake API: retries, budgets, key safety, output contracts	28
Admin page and service launcher	API security, key handling, decisions, launched services, Windows regressions, the page script	62
Documentation	generated parts current, links and files valid, diagrams well-formed	11
LangGraph-style explainer	the small comparison version	9
The generated service's own suite	the tests the pipeline gates generated code with (each runs as its own case)	32

The three scenarios, each run for real while this document was generated.

Scenario	Decomposition and orchestration	Validation result
Greenfield (URL shortener)	9 stages in 7 waves; tests, docs and UI run in parallel and join at review; 2 human approvals	completed; the generated service passes its 32 tests (0 failing)
Brownfield (BUG-142)	8 stages; impact analysis first, then a patch judged against the baseline	31 existing tests pass on the buggy code; the new regression tests (3) fail on it, proving the bug; 34 pass after the fix; 3 symbols changed
Ambiguous ("make links expire sensibly")	10 stages; 3 decisions, 1 answered by a human	completed; changing one answer afterwards redid 6 of 10 stages, and the run completed again

Faults. Every scenario has scripted defects (a leaked secret, a disallowed import, a plan with a cycle, a patch that breaks the public API, a decision the code ignores, and more). Each is injected into an agent's output and each is stopped by a named

control (full table in architecture-reference.md). After each change, the guards it relied on were also deliberately broken to confirm the tests fail; the results are recorded per task in ai-usage-log.md.

What running it for real found: defects only real use exposed, all fixed and covered by regression tests.

Found by	Defect	Fix
Running the ambiguous scenario	The command line passed answers as text, so a consistency rule crashed	Answers are typed; the policy and the engine now fail closed
Admin-page tests	A decision's id was overwritten by the question's own id (an answer would have gone nowhere); a run waiting on an approval kept the process alive after Ctrl+C	Separate ids; shutdown releases waiting decisions
The author's Windows machine	Stopping a service killed only the launcher, leaving the real service running with its port open	Stop the whole process tree
The author's Windows machine	"Start this service" failed with the default relative folder	Absolute paths for every child process
The author's Windows machine	A rate-limited request arrived as "connection reset"	Both servers read the request body before replying
The author's live runs (Sonnet)	The model was never told the 600-line limit, which its 687-line service then broke	Limits are stated in the prompts, settable per run and audited
The author's live runs (Sonnet)	The reviewer rejected three times, reasons hidden; it never saw the request, and a retry made it repeat itself	Reviewer sees the request and what was verified; reasons shown; a retry re-checks
The author's live runs (Sonnet)	No way to start the service after the first successful run: "runnable" meant "has our reference's function names"	Judged by behaviour; the page says why when it cannot offer one
The author's live runs (Sonnet)	The reviewer found a **real** defect in the model's code (non-ASCII URLs crash the redirect) and the run halted instead of fixing it	A rejection is sent back to the stages that produced the work: bounded, checkpointed, audited

5. Risks and trade-offs

Risk	Mitigation	Residual
A real model's output is worse than the stored mock output, so gates reject it and runs halt	Output contracts, the rejection fed back on retry, review findings sent back for bounded rework, call and token budgets, hybrid mode, recordings for tuning	One complete live run, after four fixes; a single run is not a reliability measure
A control misses something, so unsafe code passes	Named controls that fail closed, defence in depth (release re-verifies; the launcher re-runs the security policies), 24 fault scenarios	Static checks only; no independent security review
Generated code runs with the user's permissions (launched service, test sandbox)	Loopback forced, credentials scrubbed from its environment, its own folder, confirmation for unreviewed code, subprocess with a timeout	Not a container; outbound network and file access are not restricted
Key exposure or runaway cost	Key in memory only and never logged or echoed, call and token budgets, explicit consent	The user must supply a limited key
Approvals become rubber stamps	Approvals show the content; only a person can answer a high-risk question; decisions are audited	Careful reading cannot be enforced
A crash loses a run in progress	Finished runs and the audit log are on disk	No resume
Behaviour differs across platforms	Real Windows runs; regression tests that reproduce the cause on any OS	macOS, other browsers and load were not tested

Trade-offs accepted. Wave scheduling (a slow stage delays its siblings' successors); in-memory run state (no resume); mock-first (fast and free, but it proves governance, not generation quality); the standard library only (nothing to install, more code

to own).

6. Assumptions

- The shortener is the *vehicle*; the subject is the orchestration layer, so the shortener is deliberately modest (one machine, SQLite).
- One trusted operator on their own machine (no multi-user or internet exposure).
- "Production-grade" is read as engineering discipline (tests, controls, observability, documentation), not as a production deployment.
- The scenarios are fixed, representative cases (BUG-142 and FEAT-207), chosen to exercise impact analysis and human decisions.
- Requirements are written in English; Python 3.10 or later; Windows or Linux; Node.js 18 or later is optional (browser-script tests only).

7. Limitations and next steps

- **Agents:** mock unless a real model is chosen; one complete live run succeeded after four fixes, which is not a measure of reliability.
- **Engine:** wave scheduling, run state in memory, one run at a time, one machine.
- **Governance:** pattern and AST analysis only, and Python only for brownfield impact analysis; not a replacement for a security tool or review.
- **The shortener:** `HEAD` requests to a short link return 405; no authentication; the rate limiter is per process and in memory; the UI shows "Never" for the server's default expiry in the ambiguous scenario; no machine-readable API definition.
- **Testing:** no load tests; the web UI was exercised by a script harness and by hand, not across browsers.
- **Next:** persisted state with resume, an event-driven scheduler, a containerised sandbox, live runs to tune prompts, an OpenAPI definition, authentication, and an externally anchored audit trail.

8. How AI was used, and how its output was checked

Who did what. Claude (Anthropic) wrote essentially all of the code, tests and documents and ran its own checks in a Linux sandbox. The author set the direction and priorities, chose between the options Claude proposed, ran the code and the test suite on a Windows machine, used the admin page, reported every failure, and carried out the git and GitHub work. The brief invites AI assistance; the per-task record is ai-usage-log.md.

How the output was checked.

- A change was accepted only after tests or a runnable demonstration exercised it, and the guards it relied on were deliberately broken to confirm the tests fail.
- The author's runs on Windows were the real-world check, and they caught what the sandbox could not.
- Documents were generated from the code wherever they could drift, and tests compare them with the code.

What went wrong, openly.

- *Bugs in AI-written code* were found by tests or real runs: a metric counted twice, an empty `<title>` accepted by a UI policy, a revision that ignored nodes restored by a rollback, a fallback agent that was not told why the primary failed, and the defects in section 4.
- *Process mistakes:* the AI tested only on Linux at first, so three Windows-only bugs reached the author's machine; one first fix was incomplete (oversized request bodies were only partly read) and was caught on a second look; a documentation sentence described a metric the code does not define and was removed; the usage log mixed voices and was rewritten to attribute each check.
- *Limits of the AI:* it cannot call the live API, so it only found the live-run gaps after the author reported them (section 4); it cannot reproduce a Windows connection reset, so the tests check the cause directly.

What assurance this gives, and what it does not. Confidence comes from tests, deliberate breakage, running the system, and generated documentation that is tested against the code. It does not come from an independent line-by-line code review or a security audit, and it should not be read as one.